

Commodore 1541 ROM genealogy

Author: Audioniek

Revision: November 3rd, 2025

Over time quite a lot of 1541 ROM revisions have taken place. This text analyzes them in a chronological order. As the 1541 was in fact a development of the V(I)C-1540¹ and that drive in turn was in effect a 2031 with a serial bus interface instead of an IEEE-488 connection, the sequence starts with the 2031, the father of all single drive Commodore floppy drives with one CPU.

Checksum bytes

At the time the various ROMs were produced, Commodore used a mechanism based on a checksum to verify the code/ROM integrity. In all 1541 variants, these checksums are checked upon power up, and the drive will not start and show a flashing LED in case a checksum is found to be wrong.

The algorithm to determine the checksum is simple. All the bytes in the ROM (including the checksum byte) are added together, with a possible carry resulting from the previous addition used for the next addition. After adding all the ROM bytes together the final potential carry is processed by adding zero to the current result. The resulting 8-bit checksum should be equal to the high byte address of the first byte in the ROM; for example an 8k byte ROM intended for the addresses \$C000-\$DFFF should have a checksum of \$C0.

The checksum byte for the ROM at \$C000-\$DFFF is located at \$C000; that for the ROM at \$E000-\$FFFF at \$FEE6.

ROMs for the 1541B and the 1541-II always have the value \$79 at address \$FEE6 and \$97 at \$C000, and use the bytes at \$FFE5 and \$C001 for the correction of the checksum. Note that these versions only use one ROM device but still have two correction bytes. The reason is likely that Commodore did not change the power up checking routines in these models to remain as compatible as possible with previous 1541 models.

Although the checksum bytes differ for each ROM revision, they are not listed as a difference in the sections below. The values of the various ROM checksums are given in Appendix A at the end of this document.

Unused ROM space

Commodore was in the habit of filling unused areas in ROMs with \$AA bytes (disassembles as TAX). Filling unused space with \$00 or \$FF as is often done, is less advisable with the 6502 CPU, as \$00 constitutes a BRK instruction, and \$FF is an illegal opcode. \$AA at least does not cause a crash or activates stack operations. A single \$AA byte by itself surrounded by seemingly random other bytes is most likely valid code. Note that the contents of an unused area affects the ROM checksum, which is checked on power up.

Code changes

Commodore implemented changes in code by adding so called patches instead of inserting them inline and reassembling the (source) code. This technique is often done by using hand assembly, although the changes were incorporated in the assembler source code. The most probable reason is compatibility; this way the calling addresses of all routines remained unchanged between ROM revisions.

¹ In Germany the VIC-1540 was called the VC-1540 to avoid an unfortunate pun. Hence the spelling V(I)C-1540. The same goes for the V(I)C-20 computer.

In the 1541, two areas of 256 bytes each have been reserved and used as patch areas; each 8k byte ROM having one area. The areas are \$C002-\$C0FF and \$FEE6-\$FFE5. The area \$C002-\$C0FF remained unused until the second ROM revision for the 1541B² drive appeared; by then 16 kbyte ROMs were in use. The area \$FEE6-\$FFE5 was used for patches in all ROM revisions, even in the 2031.

Hardware compatibility

The various revisions of the 1541 hardware are compatible with each other but for one feature, that is unique to the 1541B model, the last version with a built-in power supply. The very small main board in this model supports a track zero sensor that can be defeated by closing a solder blob. The track zero sensor is connected to a VIA pin, that is grounded in the later model 1541-II but unconnected in the models prior to the 1541B. The net result of all this is that all ROM revisions can be used in a 1541B (this will render the track zero sensor inoperative if present) and a 1541-II, but the ROMs developed for the 1541B (251968-01 and 251968-02) cannot be used in the models prior to it (unless the VIA pin involved is connected to ground). The ROMs for the 1541B model can be identified by the fact that they bump the drive mechanism on power up.

The 2031 drive

The 2031 is in effect a single drive 4040. Commodore simplified the 4040 by omitting one drive mechanism, halving the amount of RAM and leaving out the twin CPU arrangement (one CPU as disc/mechanism controller and one for the DOS and bus interface) to reduce cost. This led to a new piece of code called the Low Cost Controller or LCC, and a new DOS version number: V2.6. The former DOS/interface CPU now does all the work and is in effect running a multitasking environment through the clever use of interrupts and the overflow flag. The 2031 writes the exact same disc format as the 4040 and has the same DOS bugs (the save-with-replace bug among them). By the time the 1541 appeared, Commodore created the 2031 LP that uses the same cabinet style as the 1541 including the colour and the low profile drive mechanism. For the 2031(LP) only one ROM revision exists, consisting of two 8 kbyte ROMs for a total of 16 kbyte (ROMs 901484-03 and 901484-05).

The V(I)C-1540 drive

The V(I)C-20 computer introduced the serial bus³, replacing the IEEE-488 interface of the computer models before it. The bus was cheaper to implement but also was quite a bit slower. Because of a bug in the 6522 VIA's shift register discovered at the time of development of the serial bus, the serial protocol had to be implemented completely in software, making it even slower. The V(I)C-1540 writes the exact same disc format as the 4040 and has the same bugs (the save-with-replace bug among them). For the V(I)C-1540 only one ROM revision exists, consisting of two 8 kbyte ROMs for a total of 16 kbyte (ROMs 325302-01 and 325303-01). The ROM 325302-01 would be used throughout a larger number of ROM revisions for the 1541 as well, as the code in it was not changed until the second ROM version to the 1541B appeared. (The 1541B and 1541-II no longer use two 8 kbyte ROMs but one 16 kbyte device.)

Compared to the 2031 most code is different in that it is located on different locations. For the DOS part however, the code is functionally the same; likely the reason Commodore did not change the DOS version number. Part of the code differences are of course caused by the difference in interface. For these reasons, a direct comparison on the byte level is not useful or instructive. If you want more info, compare the source code for the two models that can be found on the internet.

² Commodore's source code and technical documentation consistently calls this model the 1541B because for Commodore's engineering department, the model was the successor of the 1541A(2) with the short board. However the user manual refers to this model as the 1541C, probably because the matching computer is called 64C. Most sources use the designation 1541C. The type plate on the device simply states 1541 without a letter.

³ The serial bus inferred to in this document is sometimes also referred to as the IEC-bus. In German speaking countries the term IEC-bus is also used for the older IEEE-488 bus. To avoid confusion, the term serial bus is used in this document to refer to the connection between the 1541 and the computer and the name IEC-bus is avoided.

1541, 901229-01 ROM

This is the first ROM revision for the 1541. Commodore's plan was to use the 1540 for the 64 computer as well, but the internal timing of the 64 made its serial bus slower than that of the V(I)C-20; the 64 could not keep up with the floppy drive during read operations. For this reason the serial bus of the drive was slowed down, giving rise to a replacement of the ROM 325303-01 by ROM 901229-01. To keep the two versions apart the model number of the drive was changed to 1541.

For reasons unknown to the author, the ROM had additional changes introducing a bug/feature. In the format routine of the LCC, a superfluous INX instruction was added (or not replaced with a NOP), causing the data part of a newly formatted sector to contain one \$4D byte and 255 \$01 bytes, as opposed to 256 zero bytes previously. Apart from this difference, the 1541 with ROM 901229-01 writes the same format as the previous drive models 4040, 2031(LP) and V(I)C-1540 and the discs are therefore read and write compatible between these models.

All changes in ROM code were limited to the ROM for the area \$E000-\$FFFF (ROMs 325303-01 and 901229-01 respectively), the ROM 325302-01 remained unchanged (that ROM remained unchanged until Commodore switched to a single 23128 ROM for the 1541B; all upcoming changes for the original 1541 model were limited to the ROM 901229-0X).

Code differences compared to the V(I)C-1540

1540 ROM: 325303-01

1541 ROM: 901229-01

Difference #1: Power up string at \$E5B6.

Note that the V(I)C-1540 identifies itself as V170, not 1540.

E5B6: 73	.BY	\$73	;error number	E5B6: 73	.BY	\$73	;error number
E5B7: C3	.BY	' C '	!\$80	E5B7: C3	.BY	' C '	!\$80
E5B8: 42	.BY	' B '		E5B8: 42	.BY	' B '	
E5B9: 4D	.BY	' M '		E5B9: 4D	.BY	' M '	
E5BA: 20	.BY	' '		E5BA: 20	.BY	' '	
E5BB: 44	.BY	' D '		E5BB: 44	.BY	' D '	
E5BC: 4F	.BY	' O '		E5BC: 4F	.BY	' O '	
E5BD: 53	.BY	' S '		E5BD: 53	.BY	' S '	
E5BE: 20	.BY	' '		E5BE: 20	.BY	' '	
E5BF: 56	.BY	' V '		E5BF: 56	.BY	' V '	
E5C0: 32	.BY	' 2 '		E5C0: 32	.BY	' 2 '	
E5C1: 2E	.BY	' . '		E5C1: 2E	.BY	' . '	
E5C2: 36	.BY	' 6 '		E5C2: 36	.BY	' 6 '	
E5C3: 20	.BY	' '		E5C3: 20	.BY	' '	
E5C4: 56	.BY	' V '		E5C4: 31	.BY	' 1 '	
E5C5: 31	.BY	' 1 '		E5C5: 35	.BY	' 5 '	
E5C6: 37	.BY	' 7 '		E5C6: 34	.BY	' 4 '	
E5C7: B0	.BY	' 0 '	!\$80	E5C7: B1	.BY	' 1 '	!\$80

Difference #2: Slowing the serial bus for the Commodore 64.

The Commodore 64 was too slow to keep up with the V(I)C-1540. Commodore "solved" this by making the serial bus go slower instead of fixing it in the 64 computer (this is bad practice: the problem is fixed by circumventing the consequences rather than taking care of the root cause). The slowing down has been made switchable through the commands UI- and UI+ and defaults to the slow Commodore 64 speed.

Note the four NOPs in the V(I)C-1540 ROM; what was there originally or is it a deliberate delay that was extended?

```
E973: 209CE9  ISRHI  JSR DATHI
E976: 20B7E9  ISRCLK JSR CLKHI
E979: EA      NOP
E97A: EA      NOP
E97B: EA      NOP
E97C: EA      NOP
E97D: 20AEE9  JSR CLKLOW
E980: 209CE9  JSR DATHI
E983: C698    DEC CONT    ;count bits
```

```
FEF3: AA      TAX
FEF4: AA      TAX
FEF5: AA      TAX
FEF6: AA      TAX
FEF7: AA      TAX
FEF8: AA      TAX
FEF9: AA      TAX
FEFA: AA      TAX
```

```
FEFB: AA      TAX
FEFC: AA      TAX
FEFD: AA      TAX
FEFE: AA      TAX
FEFF: AA      TAX
FF00: AA      TAX
```

```
E973: 209CE9  ISRHI  JSR DATHI
E976: 20B7E9  ISRCLK JSR CLKHI
E979: A523    LDA DRVTRK+1; ! added: get flag
E97B: D003    BNE ISR03  ;! added: if zero

E97D: 20F3FE          JSR SLOWD    ;! added: wait a bit
E980: 20FBFE  ISR03  JSR CLKDAT    ;! changed: CLK lo/DATA hi
E983: C698    DEC COUNT    ;count bits
```

```
;*****
;* Patch to slow down serial bus      *
;*****
```

```
FEF3: 8A      SLOWD  TXA          ;save X
FEF4: A205    LDX #5          ;get loop count
```

```
FEF6: CA      SLOWE  DEX          ;wait a bit
FEF7: D0 FD    BNE SLOWE      ;if not done, loop
```

```
FEF9: AA      TAX          ;else restore X
FEFA: 60      RTS          ;and exit
```

```
;*****
;* Patch: set data hi when setting    *
;* clock low                          *
;*****
```

```
FEFB: 20AEE9  CLKDAT JSR CLKLOW    ;set CLK lo
```

```
FEFE: 4C9CE9          JMP DATHI    ;and DATA hi
```

Difference #3: The data block contents of a freshly formatted sector.

Why this phenomenon was introduced or somebody forgot to incorporate an already known fix is unclear. What is also unclear is why the bug was never fixed throughout the lifespan of the 1541. The bug provides an easy way to determine whether a disc was formatted in a 1541 or other drive though.

```
FC7A: EE2806      INC SECT    ;increment sector number
FC7D: AD2806      LDA SECT    ;get it
FC80: C543        CMP SECTR   ;if whole track done,
FC82: 90BB        BCC MAK10   ;proceed
FC84: 98          TYA         ;else get block size in A
FC85: 48          PHA         ;save it
FC86: EA          NOP         ;X=0 (probably a fix)
```

```
FC87: 8A          TXA         ;A=0
FC88: 9D0005      CNTDAT STA BUFF2,X ;store A in data block
FC8B: E8          INX         ;point to next
FC8C: D0FA        BNE CNTDAT  ;loop until block filled
```

```
FC7A: EE2806      INC SECT    ;increment sector number
FC7D: AD2806      LDA SECT    ;get it
FC80: C543        CMP SECTR   ;if at maximum sector#,
FC82: 90BB        BCC MAK10   ;proceed
FC84: 98          TYA         ;else get block size in A
FC85: 48          PHA         ;save it
FC86: E8          INX         ;! BUG: X should be 0!
                                ;(is one now)
```

```
*****
;* BUG: because X holds drive# (zero at *
;* this point) it is incremented to  *
;* one.                                *
;*                                    *
;* This has the following effect:      *
;*                                    *
;* - The first byte of the data block  *
;*   is skipped (and left at $4D);     *
;* - Because X is 1, the remaining data *
;*   bytes in the block are set to $01. *
;*                                    *
;* The 2031/4040/1540 fill all 256 data *
;* bytes with the value zero.          *
;*                                    *
*****
```

```
FC87: 8A          TXA         ;move X to A (A=1)
FC88: 9D0005      CNTDAT STA BUFF2,X ;fill data part0
FC8B: E8          INX         ;point to next
FC8C: D0FA        BNE CNTDAT  ;loop until block filled
```

Difference #4: Make the serial bus speed switchable.

The disc command UI (or U9) unleashes the same actions as an actual NMI and its associated processing. The NMI vector was changed and points to a new routine that tests for plus and minus characters as third command character. If not found, the original NMI code is executed.

```
FF01: AA      TAX
FF02: AA      TAX
FF03: AA      TAX
FF04: AA      TAX
FF05: AA      TAX
FF06: AA      TAX
FF07: AA      TAX
FF08: AA      TAX
FF09: AA      TAX
FF0A: AA      TAX
FF0B: AA      TAX
FF0C: AA      TAX
FF0D: AA      TAX
FF0E: AA      TAX
FF0F: AA      TAX
```

```
;*****
;* 6502 vectors *
;*****
```

```
FFFA: E7FE      .WO NMI      ;NMI
FFFC: A0EA      .WO DSKINT   ;Reset
FFFE: 67FE      .WO IRQ      ;IRQ
```

```
;*****
;* NMI entry. *
;* *
;* Check for UI+ and UI- to set serial *
;* bus speed. *
;* *
;*****
FF01: AD0202 NNMI LDA CMDBUF+2 ;get 3rd command char.

FF04: C92D      CMP # '-' ;if a minus

FF06: F005      BEQ NNMI10 ;set $23 to non zero

FF08: 38        SEC          ;else prepare for subtract
FF09: E92B      SBC # '+' ;if original was a plus

FF0B: D0DA      BNE NMI      ;zero $23

FF0D: 8523      NNMI10 STA DRVTRK+1

FF0F: 60        RTS          ;and exit
```

```
;*****
;* 6502 vectors *
;*****
```

```
FFFA: 01FF      .WO NNMI     ;! changed
FFFC: A0EA      .WO DSKINT   ;Reset
FFFE: 67FE      .WO IRQ      ;IRQ
```

This concludes the differences between the V(I)C-1540 325303-01 and 1541 901229-01 ROMs.

1541, 901229-02 ROM

The second ROM revision for the 1541 only brought one serial protocol change.

All changes in ROM code were limited to the ROM for the area \$E000-\$FFFF (ROM 901229-02). The changes that occurred between ROMs 325303-01 and 901229-01 were retained unchanged.

Code differences compared to the ROM 901229-01

ROMs involved: 901229-01 and 901229-02

Difference #1: Change reading a data bit.

The change swaps the order of events while reading the next bit of a byte off the serial bus: the reading of the bit and waiting for the CLK line to go low have been reversed. The NOPs are there because one read of VIA port B is now superfluous. The comments in the source code refer to a removal of a (non-existent) speed up for the European V(I)C-20, which seems odd, as the new code is faster (measured in clock cycles).

E9F2:	20A5E9	ACP00B	JSR DATLOW	;set DATA low	E9F2:	20A5E9	ACP00B	JSR DATLOW	;set DATA low
E9F5:	A20A		LDX #10	;wait	E9F5:	A20A		LDX #10	;wait
E9F7:	CA	ACP02	DEX	;a	E9F7:	CA	ACP02	DEX	;a
E9F8:	D0FD		BNE ACP02	;bit	E9F8:	D0FD		BNE ACP02	;bit
E9FA:	209CE9		JSR DATHI	;set DATA high	E9FA:	209CE9		JSR DATHI	;set DATA high
E9FD:	2059EA	ACP02A	JSR TSTATN	;test for ATN	E9FD:	2059EA	ACP02A	JSR TSTATN	;test for ATN
EA00:	20C0E9		JSR DEBNC	;wait for CLK low	EA00:	20C0E9		JSR DEBNC	;wait for CLK low
EA03:	2904		AND #\$04	;get CLK in	EA03:	2904		AND #\$04	;get CLK in
EA05:	F0F6		BEQ ACP02A	;if hi, wait	EA05:	F0F6		BEQ ACP02A	;if hi, wait
EA07:	A900		LDA #\$00	;else	EA07:	A900		LDA #\$00	;else
EA09:	85F8		STA EOIFLG	;clear EOI flag	EA09:	85F8		STA EOIFLG	;clear EOI flag
EA0B:	AD0018	ACP01	LDA PB	;get bus lines	EA0B:	AD0018	ACP01	LDA PB	;get bus lines
EA0E:	2904		AND #\$04	;get CLK in	EA0E:	4901		EOR #\$01	;! changed: toggle DATA in
EA10:	D0F9		BNE ACP01	;if lo, wait	EA10:	4A		LSR A	;! changed: get bit in C
EA12:	AD0018		LDA PB	;get bus lines	EA11:	2902		AND #\$02	;! changed: wait for CLK
EA15:	4901		EOR #\$01	;invert DATA in	EA13:	D0F6		BNE ACP01	;! changed: hi
EA17:	4A		LSR A	;get it in C	EA15:	EA		NOP	;! changed: remove PAL
					EA16:	EA		NOP	;! changed: V(I)C-20
EA18:	6685		ROR DATA	;rotate received bit in	EA17:	EA		NOP	;! changed: speedup
EA1A:	2059EA	ACP03A	JSR TSTATN		EA18:	6685		ROR DATA	;rotate received bit in
					EA1A:	2059EA	ACP03A	JSR TSTATN	

Difference #2: Initials of the programmer added.

At the end of the ROM the letters RSR were added, probably standing for the initials of the author of the difference mentioned above.

FFE1:	AA	TAX	FFE1:	AA	TAX
FFE2:	AA	TAX	FFE2:	52	.BY ' R ' ;! changed: patcher's
FFE3:	AA	TAX	FFE3:	53	.BY ' S ' ;! changed: initials
FFE4:	AA	TAX	FFE4:	52	.BY ' R ' ;! changed
FFE5:	AA	TAX	FFE5:	AA	TAX

This concludes the differences between the 1541 901229-01 and 901229-02 ROMs.

1541, 901229-03 ROM

The third ROM revision for the 1541 only changed the value of gap1 in the sector headers.

All changes in ROM code were limited to the ROM for the area \$E000-\$FFFF (ROM 901229-03). The changes that occurred between ROMs 901229-01 and 901229-02 were retained unchanged.

Code differences compared to the ROM 901229-02

ROMs involved: 901229-02 and 901229-03

Difference #1: Change gap1.

The change makes gap1 (the number of disc bytes between two parts of a sector header) one disc byte (10 bits) longer. The change is required in two places: the code that (re)writes a sector starting at \$F586 and the formatting code starting at \$FCCD. In both cases only one constant is changed.

This change causes discs formatted with all previous ROMs (as well as the 2031(LP) and the 4040) to be no longer write compatible. Because the read routines simply wait during the passing of the gap for the SYNC marker to pass, reading is not affected: the new value for gap1 simply causes the remaining sector parts to pass the head later in time and within the time slot allowed.

F586:	208FF7	WRT10	JSR BINGCR	;convert the data to GCR	F586:	208FF7	WRT10	JSR BINGCR	
F589:	2010F5		JSR SRCH	;position the head	F589:	2010F5		JSR SRCH	
F58C:	A209		LDX #8	;get gap1 length	F58C:	A209		LDX #9	;! changed: get gap1
F58E:	50FE	WRT20	BVC WRT20		F58E:	50FE	WRT20	BVC WRT20	
F590:	B8		CLV		F590:	B8		CLV	
F591:	CA		DEX	;count gap bytes	F591:	CA		DEX	;count gap bytes
F592:	D0FA		BNE WRT20	;and loop	F592:	D0FA		BNE WRT20	;and loop
FCCD:	D0F3		BNE WRTS20		FCCD:	D0F3		BNE WRTS20	
FCCF:	A209		LDX #8	;get gap1 length	FCCF:	A209		LDX #9	;! changed: get gap1 count
FCD1:	50FE	WRTS30	BVC WRTS30		FCD1:	50FE	WRTS30	BVC WRTS30	
FCD3:	B8		CLV		FCD3:	B8		CLV	
FCD4:	A955		LDA #\$55	;get gap data	FCD4:	A955		LDA #\$55	;get gap data
FCD6:	8D011C		STA DATA2	;set it	FCD6:	8D011C		STA DATA2	;set it
FCD9:	CA		DEX	;count gap bytes	FCD9:	CA		DEX	;count gap bytes
FCDA:	D0F5		BNE WRTS30	;and loop	FCDA:	D0F5		BNE WRTS30	;and loop
				; write out data block					; write out data block

This concludes the differences between the 1541 901229-02 and 901229-03 ROMs.

1541, 901229-04 ROM

The fourth ROM revision for the 1541 was apparently only used for the internal drive mounted in the SX-64 portable computer. It improved power on behaviour, and improved handling an EOI condition.

All changes in ROM code were limited to the ROM for the area \$E000-\$FFFF (ROM 901229-04). The changes that occurred between ROMs 901229-02 and 901229-03 were retained unchanged apart from difference #4 below.

Code differences compared to the ROM 901229-03

ROMs involved: 901229-03 and 901229-04

Difference #1: Wait for all devices on the serial bus to idle before starting the EOI timer.

```
E9CD: 2059EA  ACP00A JSR TSTATN
E9D0: 20C0E9          JSR DEBNC
E9D3: 2904          AND #$04    ;get state of CLK
E9D5: D0F6          BNE ACP00A  ;wait for high
E9D7: 209CE9          JSR DATHI  ;then set DATA high
E9DA: A901          LDA #$01    ;wait
E9DC: 8D0518          STA T1HC1  ;255 us
```

```
E9DF: 2059EA  ACP00 JSR TSTATN
```

```
FF20: AA          TAX
FF21: AA          TAX
FF22: AA          TAX
FF23: AA          TAX
FF24: AA          TAX
FF25: AA          TAX
FF26: AA          TAX
FF27: AA          TAX
FF28: AA          TAX
FF29: AA          TAX
FF2A: AA          TAX
FF2B: AA          TAX
FF2C: AA          TAX
FF2D: AA          TAX
FF2E: AA          TAX
```

```
E9CD: 2059EA  ACP00A JSR TSTATN
E9D0: 20C0E9          JSR DEBNC
E9D3: 2904          AND #$04    ;get state of CLK
E9D5: D0F6          BNE ACP00A  ;wait for high
E9D7: 209CE9          JSR DATHI  ;then set DATA high
E9DA: A901          LDA #$01    ;wait
E9DC: 4C20FF          JMP PATCH6 ;! changed: wait for all
                                         ;devices to idle, start
                                         ;timer for 255 us
```

```
E9DF: 2059EA  ACP00 JSR TSTATN
```

Added:

```
                                         ;*****
;* PATCH6:                               *
;*                                         *
;* Fix EOI behaviour to wait for all    *
;* devices                               *
;*                                         *
                                         ;*****
FF20: AD0018  PATCH6 LDA PB    ;get bus lines

FF23: 2901          AND #$01    ;get DATA in

FF25: D0F9          BNE PATCH6  ;wait for it to go high
                                         ;(all devices idle)
FF27: A901          LDA #$01    ;now

FF29: 8D0518          STA T1HC1  ;set timer to

FF2C: 4CDFE9          JMP ACP00  ;wait 255 us and return
```

Difference #2: On power on, immediately set CLK line hi.

This change immediately sets the bus CLK line to high on power up, to avoid blocking the serial bus until the whole drive initialization is completed.

```
*****
;* Reset entry *
*****
EAA0: 78    DSKINT SEI        ;no interrupts
EAA1: D8          CLD          ;clear decimal mode
EAA2: A2FF       LDX #$FF      ;direction is all outputs
EAA4: 8E0318     STX DDRA1     ;set it on port A
EAA7: E8          INX          ;X=0
```

```
FF10: AA          TAX
FF11: AA          TAX
FF12: AA          TAX
FF13: AA          TAX
FF14: AA          TAX
FF15: AA          TAX
FF16: AA          TAX
FF17: AA          TAX
FF18: AA          TAX
FF19: AA          TAX
FF1A: AA          TAX
FF1B: AA          TAX
FF1C: AA          TAX
FF1D: AA          TAX
FF1E: AA          TAX
FF1F: AA          TAX
```

```
*****
;* Reset entry *
*****
EAA0: 78    DSKINT SEI        ;no interrupts
EAA1: D8          CLD          ;clear decimal mode
EAA2: A2FF       LDX #$FF      ;direction is all outputs
EAA4: 4C10FF     JMP PATCH5    ;! changed: set CLK out hi
EAA7: E8    DKIT10 INX          ;X=0
```

Added:

```
*****
;* PATCH5: *
;* *
;* Set CLK hi *
;* *
*****
FF10: 8E0318 PATCH5 STX DDRA1 ;set direction of port A

FF13: A902          LDA #$02    ;set CLK out high

FF15: 8D0018        STA PB      ;on bus

FF18: A91A          LDA #$1A    ;set DDR port B
FF1A: 8D0218        STA DDRB1

FF1D: 4CA7EA        JMP DKIT10  ;and return
```

Note on PATCH5: The setting of the CLK out line is done by first writing the desired value and then setting the port direction although this seems not to be the logical order. This approach avoids having a temporarily unknown state on the output pins involved however (also see difference #3 below).

Difference #3: Improve the initialization of the VIA port controlling the bus lines on power on.

This change simply reverses the order of setting the port data and the associated direction register. The new order is setting the data first, then setting the line directions. This has the advantage that the port lines do not temporarily assume an undetermined value in the new situation; they immediately assume the desired state upon setting the direction.

EBCD: A90A	LDA #10	;set up	EBCD: A90A	LDA #10	;set up
EBCF: 8569	STA SECINC	;vector offset	EBCF: 8569	STA SECINC	;vector offset
EBD1: A905	LDA #55	;set up	EBD1: A905	LDA #5	;set up
EBD3: 856A	STA REVCNT	;recovery count	EBD3: 856A	STA REVCNT	;recovery count
EBD5: A973	LDA #\$73	;get power on error code	EBD5: A973	LDA #\$73	;get power on error code
EBD7: 20C1E6	JSR ERRTS0	;set it	EBD7: 20C1E6	JSR ERRTS0	;set it
EBDA: A91A	LDA #\$1A	;get direction bits	EBDA: A900	LDA #\$00	;! changed: first set
					;lines all hi
EBDC: 8D0218	STA DDRB1	set direction	EBDC: 8D0018	STA PB	;! changed: then
EBDF: A900	LDA #\$00	;set	EBDF: A91A	LDA #\$1A	;! changed: set
EBE1: 8D0018	STA PB	;bus lines	EBE1: 8D0218	STA DDRB1	;! changed: direction
EBE4: 2080E7	JSR B00T	;check for diagnostic, ;if not:	EBE4: 2080E7	JSR B00T	;check for diagnostic, ;if not:
		;Idle loop			; Idle loop
EBE7: 58	IDLE CLI	;no interrupts	EBE7: 58	IDLE CLI	;no interrupts

Difference #4: Removal of the initials of the programmer.

The letters RSR at the end of the ROM (introduced with ROM 901229-02) were removed again.

FFE1: AA	TAX	FFE1: AA	TAX
FFE2: 52	.BY ' R ' ;patcher's	FFE2: AA	TAX
FFE3: 53	.BY ' S ' ;initials	FFE3: AA	TAX
FFE4: 52	.BY ' R '	FFE4: AA	TAX
FFE5: AA	TAX	FFE5: AA	TAX

This concludes the differences between the 1541 901229-03 and 901229-04 ROMs.

1541, 901229-05 ROM

The fifth ROM revision for the 1541 removed some remnants of the IEEE-488 interface of the 2031. In addition the power-on diagnostic sensing was removed, as its functionality was already largely covered by the UTLODR code.

All changes in ROM code were limited to the ROM for the area \$E000-\$FFFF (ROM 901229-05). The changes that occurred between ROMs 901229-03 and 901229-04 were retained unchanged.

Of the ROM revisions for the 1541 with built-in power supply, this is probably the most common.

Code differences compared to the ROM 901229-04

ROMs involved: 901229-04 and 901229-05

Difference #1: Remove IEEE-488 remnants.

		; Talker error recovery				; Talker error recovery
		;				;
		; If command channel, release DAV (n.a.)				
		; If data channel, force not ready				
		; (n.a.) and release channel				
E680:	20EBD0	TLKERR JSR FNDRCH	;find read channel	E680:	20EBD0	TLKERR JSR FNDRCH ;find read channel
E683:	204EEA	JSR ITERR	;release bus lines	E683:	EA	NOP ;! changed: no JSR ITERR
			;then go to idle	E684:	EA	NOP
				E685:	EA	NOP
E686:	D006	BNE TLERR	;(never executed)	E686:	D006	BNE TLERR ;(is executed)
		; Listener error recovery				; Listener error recovery
		;				;
		; If command channel, release RFD (n.a.)				
		; If data channel, force not ready (n.a.) and				
		; release channel				
E688:	2007D1	LSNERR JSR FNDWCH	;find write channel	E688:	2007D1	LSNERR JSR FNDWCH ;find write channel
E68B:	204EEA	JSR ILERR	;release bus lines	E68B:	EA	NOP ;! changed: no JSR ILERR
			;then go to idle	E68C:	EA	NOP
				E68D:	EA	NOP
E68E:	2025D1	TLERR JSR TYPFIL	;(never executed?)	E68E:	2025D1	TLERR JSR TYPFIL ;get file type (executed)
E691:	C904	CMP #\$04	;if relative	E691:	C904	CMP #\$04 ;if relative
E693:	B003	BCS ERR10		E693:	B003	BCS ERR10

Notes:

Although they have different labels, the routines ITERR and ILERR are one and the same. The routine itself was not removed from the ROM, as it is called in other parts of the code as well.

Both the original and the patched code have some anomalies. The routine ITERR/ILERR does not end with an RTS, but always jumps to the idle loop. The BNE instruction at \$D006 is therefore never executed in the original code; as a result the code at label TLERR (\$E68E) is never executed when calling either TLKERR (\$E680) or LSNERR (\$E688).

The patching however leaves the BNE at \$D006 in, and the instruction now tests the result of the routine FNDRCH when calling TLKERR (\$E680) using the zero flag. This routine however uses the carry flag to report its result and the state of the zero flag is not consistent upon the exit of this routine. The net result is that the behaviour of the code before and after the change can be totally different to say the least.

The same goes for a call to LSNERR (\$E688). A call of the original code would eventually enter the idle loop, but after patching it falls through to TLERR. Again, net result is that the behaviour of the code before and after the change can be totally different.

TLERR is used in the preceding code; to restore the same behaviour before and after the patch, three NOPs should be replaced with a JMP IDLE (\$4C \$E7 \$EB, two instances).

Further analysis is beyond the scope of this document.

Difference #2: Remove diagnostic sensing code.

The 1540 and 1541 can be put in a diagnostic mode by externally grounding both the CLK and DATA serial bus lines and switching the drive on. After removal of the connections the UTLODR code is used to load a file over the bus. Upon completion of the load and a successful checksum check the loaded file is executed. The diagnostic sense part was removed with this ROM revision, the UTLODR was retained however. Note that the routine was replaced by an RTS instruction and NOPs but for the last byte of the routine: this was also replaced by an RTS instruction.

; Power on diagnostic sensing					; Power on diagnostic sensing				
E77F:	60	BOOT2	RTS	;exit	E77F:	60	BOOT2	RTS	;exit
E780:	AD0018	BOOT	LDA PB	;get bus lines	E780:	60	BOOT	RTS	;! immediately return
					E781:	EA		NOP	;! changed
					E782:	EA		NOP	;! changed
E783:	AA		TAX	;in X	E783:	EA		NOP	;! changed
E784:	2904		AND #\$04	;get CLK in	E784:	EA		NOP	;! changed
					E785:	EA		NOP	;! changed
E786:	F0F7		BEQ BOOT2	;if hi, exit	E786:	EA		NOP	;! changed
					E787:	EA		NOP	;! changed
E788:	8A		TXA	;get bus lines back	E788:	EA		NOP	;! changed
E789:	2901		AND #\$01	;get DATA in	E789:	EA		NOP	;! changed
					E78A:	EA		NOP	;! changed
E78B:	F0F2		BEQ BOOT2	;if hi, exit	E78B:	EA		NOP	;! changed
					E78C:	EA		NOP	;! changed
E78D:	58		CLI	;if both CLK and DATA low, ;diag is set, ;allow interrupts	E78D:	EA		NOP	;! changed
					E78E:	EA		NOP	;! changed
E78E:	AD0018	BOOT3	LDA PB	;get bus lines	E78F:	EA		NOP	;! changed
E791:	2905		AND #\$05	;wait for both to go hi	E790:	EA		NOP	;! changed
					E791:	EA		NOP	;! changed
E793:	D0F9		BNE BOOT3		E792:	EA		NOP	;! changed
					E793:	EA		NOP	;! changed
E795:	EE7802		INC F2CNT	;	E794:	EA		NOP	;! changed
					E795:	EA		NOP	;! changed
					E796:	EA		NOP	;! changed
					E797:	EA		NOP	;! changed
E798:	EE7402		INC CMDSIZ	;file name length is 1	E798:	EA		NOP	;! changed
					E799:	EA		NOP	;! changed
					E79A:	EA		NOP	;! changed
E79B:	A92A		LDA # ' * '	;any file name will do	E79B:	EA		NOP	;! changed
					E79C:	EA		NOP	;! changed
E79D:	8D0002		STA CMDBUF		E79D:	EA		NOP	;! changed
					E79E:	EA		NOP	;! changed
					E79F:	EA		NOP	;! changed
E7A0:	4CA8E7		JMP BOOT4	;and go load test file	E7A0:	EA		NOP	;! changed
					E7A1:	EA		NOP	;! changed
					E7A2:	60		RTS	;! changed

This concludes the differences between the 1541 901229-04 and 901229-05 ROMs.

1541, 901229-06 ROM

The sixth ROM revision for the 1541 improved formatting by clearing the format flag before starting the actual formatting of a disc.

All changes in ROM code were limited to the ROM for the area \$E000-\$FFFF (ROM 901229-06). The changes that occurred between ROMs 901229-04 and 901229-05 were retained unchanged.

This was the last revision to appear for the 1541 models. The next revision was for the 1541B model and used a 16 kbyte ROM instead of two 8 kbyte devices.

Code differences compared to the ROM 901229-05

ROMs involved: 901229-05 and 901229-06

Difference #1: Clear format flags before formatting.

EE39:	A901	LDA #1	;start	EE39:	A901	LDA #1	;start
EE3B:	8580	STA TRACK	;with track 1	EE3B:	8580	STA TRACK	;with track 1
EE3D:	20C6C8	JSR FORMAT	;transfer format to RAM	EE3D:	202FFF	JSR PATCH7	;! changed: clear format ;flags before formatting
EE40:	2005F0	JSR CLRBAM	;create an empty BAM	EE40:	2005F0	JSR CLRBAM	;create an empty BAM
EE43:	4C56EE	JMP N110	;and go write empty BAM	EE43:	4C56EE	JMP N110	;and go write empty BAM
				;*****			
				;* PATCH7:			
				;* *			
				;* Clear format flags before formatting *			
				;* *			
				;*****			
FF2F:	AA	TAX		FF2F:	A9FF	PATCH7 LDA #\$FF	;clear
FF30:	AA	TAX					
FF31:	AA	TAX		FF31:	8551	STA FTNUM	;format flags
FF32:	AA	TAX					
FF33:	AA	TAX		FF33:	4CC6C8	JMP FORMAT	;and do old routine ;(transfer format to RAM)
FF34:	AA	TAX					

This concludes the differences between the 1541 901229-05 and 901229-06 ROMs.

1541B, 251968-01 ROM

The first ROM revision for the 1541B introduced three changes. The first is the switch to a single 16 kbyte ROM instead of two 8 kbyte devices. This enabled the use of the second patch area in the range \$C002-\$C0FF without the need for two new ROMs. The area was not used with this ROM revision.

The second change is support for a track 0 sensor⁴ to get rid of the bumping while recalibrating. To initialize the track position, the 1541B recalibrates on power up; when the track 0 sensor is present this is almost silent and does not mechanically stress the drive mechanics.

On the main board, a solder blob is present to defeat the sensor. When closed, the drive will do the classic rattling bump (also on power up). The 1541B was produced with both Newtronics and ALPS mechanics; in practice only a part of the Newtronics drives actually had a track zero sensor.

The third change is an attempt to speed up the drive a bit by servicing the command(s) to the LCC or occurrences of ATNs not every 15 milliseconds, but every 8 milliseconds.

A fourth change did not occur. Strictly speaking the check of the ROM checksum should be done over one block of 16 kbyte, as that is the actual size of the ROM. The power on routine still checks two 8 kbyte blocks as it used to and there are still two checksum correction bytes, but in different locations as before. The source code says to leave the original locations unchanged in the comments (\$97 at \$C000 and \$79 at \$FEE6). A likely reason for this is that Commodore wanted the ROM to be as compatible as possible with previous versions and therefore did not change the power up test, and also wanted away to identify a 16 kbyte single ROM version with *two* checksum correction bytes.

The changes in ROM code were still only present in the ROM area covering \$E000-\$FFFF (second half of ROM 251968-01). The changes that occurred between ROMs 901229-05 and 901229-06 were retained unchanged.

Code differences compared to the ROM 901229-06

ROMs involved: 901229-06 and 251968-01

⁴ In Commodore DOS, there is no track zero. The outermost track of a drive has the number one. The common terminology with drive mechanisms uses the term track 0 sensor for the device that signals the head is at the outermost track. For reasons of clarity, the term track 0 was retained here and should be read as "outermost track" in the context of this document.

Difference #1: Track 0 sensor support, power on initialization.

;*****				;*****			
;* RESET entry point *				;* RESET entry point *			
;*****				;*****			
EAA0:	78	DSKINT SEI	;no interrupts	EAA0:	78	DSKINT SEI	;no interrupts
EAA1:	D8	CLD	;clear decimal mode	EAA1:	D8	CLD	;clear decimal mode
EAA2:	A2FE	LDX #\$FF	;port A: all outputs	EAA2:	A2FE	LDX #\$FE	;! changed: PA0 is an ;input (TRK0)
EAA4:	4C10FF	JMP PATCH5		EAA4:	4C10FF	JMP PATCH5	
EAA7:	C8	DKIT10 INX	;X=0	EAA7:	C8	DKIT10 INX	;! X=\$FF
;*****				;*****			
;* PATCH5: *				;* PATCH5: *			
;* *				;* *			
;* Set CLK hi *				;* Set CLK hi *			
;* *				;* *			
;*****				;*****			
FF10:	8E0318	PATCH5 STX DDRA1	;set direction	FF10:	8E0318	PATCH5 STX DDRA1	;set direction
FF13:	A902	LDA #\$02	;set CLK out high	FF13:	A902	LDA #\$02	;set CLK out high
FF15:	8D0018	STA PB	;on bus	FF15:	8D0018	STA PB	;on bus
FF18:	A91A	LDA #\$1A	;set DDR port B	FF18:	A91A	LDA #\$1A	;set DDR port B
FF1A:	8D0218	STA DDRB1		FF1A:	8D0218	STA DDRB1	
FF1D:	4CA7EA	JMP DKIT10	;and return	FF1D:	4CA7EA	JMP DKIT10	;and return

Notes: PATCH5 was already present since ROM 901229-04 and is unchanged. Although register X now holds \$FF instead of 0 at address \$EAA8 this has no consequences; X is not used and two instructions later set to zero using an LDX #0 instruction in preparation of the RAM test.

Difference #2: Track 0 sensor support, perform the power on bump.

EBBA: 851D STA WPSW+1 ;clear WP status drive 1				EBBA: 851D STA WPSW+1 ;clear WP status drive 1			
EBBC: 2063CB JSR USRINT ;initialize USER JMP				EBBC: 2063CB JSR USRINT ;initialize USER JMP			
EBBF: 20FACE JSR LRUINT ;initialize LRU				EBBF: 20FACE JSR LRUINT ;initialize LRU			
EBC2: 2059F2 JSR CNTINT ;controller initialization				EBC2: 206FF2 JSR PTCH10 ;! changed: perform a bump			
EBC5: A922 LDA #<DIAGOK				EBC5: A922 LDA #<DIAGOK			
EBC7: 8565 STA VNMI				EBC7: 8565 STA VNMI			
				;*****			
				;* PTCH10: *			
				;* *			
				;* Move head to track zero at end of *			
				;* device initialization *			
				;* *			
				;*****			
FF6F:	AA	TAX		FF6F:	2059F2	PTCH10 JSR CNTINT	;controller initialization
FF70:	AA	TAX					
FF71:	AA	TAX					
FF72:	AA	TAX		FF72:	A901	LDA #\$01	;command length
FF73:	AA	TAX					
FF74:	AA	TAX		FF74:	8506	STA HDRS	;is one byte
FF75:	AA	TAX					
FF76:	AA	TAX		FF76:	A9C0	LDA #\$C0	;get bump command
FF77:	AA	TAX					
FF78:	AA	TAX		FF78:	8500	STA JOBS	;set it
FF79:	AA	TAX					
FF7A:	AA	TAX		FF7A:	60	RTS	;and exit

Difference #3: Improve operating speed slightly by setting periodic timer interrupt to every 8 milliseconds.

```
F26C: 8D0C1C      STA PCR2
F26F: A941        LDA #$41      ;cont IRQ, latch mode
F271: 8D0B1C      STA PCR2
F274: A900        LDA #$00      ;get timer1 lo byte
F276: 8D061C      STA T1LL2    ;set it
F279: A93A        LDA #$3A      ;15ms IRQ
F27B: 8D071C      STA T1HL2    ;set timer 2 hi
F27E: 8D 051C     STA T1HC2    ;get 6522's attention
F281: A97F        LDA #$7F      ;clear all IRQ sources
```

```
F26C: 8D0C1C      STA PCR2
F26F: A941        LDA #$41      ;cont IRQ, latch mode
F271: 8D0B1C      STA PCR2
F274: A900        LDA #$00      ;get timer1 lo byte
F276: 8D061C      STA T1LL2    ;set it
F279: A920        LDA #$20      ;! changed: 8ms IRQ
F27B: 8D071C      STA T1HL2    ;set timer 2 hi
F27E: 8D 051C     STA T1HC2    ;get 6522's attention
F281: A97F        LDA #$7F      ;clear all IRQ sources
```

Difference #4: Track 0 sensor support, check for track 0 while stepping out.

```
FA2E: A54A      DOSTEP LDA STEPS ;get steps to do/direction
FA30: 1031      BPL STPIN  ;if stepping in, do that
FA32: E64A      INC STEPS  ;update steps to do
FA34: AE001C    LDX DSKCNT  ;get phase
FA37: CA        DEX        ;update it

FA38: 4C69FA    JMP STP     ;and drive step motor
FA3B: A54A      SHORT LDA STEPS ;get # of steps to do
```

FF36: AA TAX

up
to

FF6C: AA TAX
FF6D: AA TAX
FF6E: AA TAX

```
FA2E: A54A      DOSTEP LDA STEPS ;get steps to do/direction
FA30: 1031      BPL STPIN  ;if stepping in, do that
FA32: 4C36FF    STPOUT JMP PATCH9 ;!: check for track 0
FA35: EA        NOP        ;! changed
FA36: EA        NOP        ;! changed
FA37: EA        NOP        ;! changed
FA38: 4C69FA    PPPPPP JMP STP     ;and drive step motor
FA3B: A54A      SHORT LDA STEPS ;get # of steps to do
```

```
*****
;* PATCH9: *
;* *
;* Check if at track 0 *
;* *
*****
```

```
FF36: 8A      PATCH9 TXA      ;save
FF37: 48      PHA      ;X
FF38: 98      TYA      ;and
FF39: 48      PHA      ;Y
FF3A: A201    LDX #$01    ;get outer loop count
FF3C: A064    PTCH91 LDY #100 ;get inner loop count
FF3E: AD0F18  PTCH92 LDA POTA1 ;get port A
FF41: CD0F18  CMP POTA1    ;if no change
FF44: D01C    BNE PTCH93
FF46: 88      DEY      ;count down (inner loop)
FF47: D0F5    BNE PTCH92  ;if inner not done, loop
FF49: CA      DEX      ;else adapt outer loop
FF4A: D0F0    BNE PTCH91  ;if not finished, loop
FF4C: 2901    AND #$01    ;else get port data
FF4E: F012    BEQ PTCH93  ;if not at track 0
FF50: AD001C  LDA DSKCNT  ;get controller phase
FF53: 2903    AND #$03    ;bits
FF55: D00B    BNE PTCH93  ;if at phase A
FF57: 68      PLA      ;exit
FF58: A8      TAY      ;restoring
FF59: 68      PLA      ;Y
FF5A: AA      TAX      ;and X
FF5B: A900    LDA #0      ;clear
FF5D: 854A    STA STEPS   ;number of steps to do
FF5F: 4CBEFA  JMP END33   ;and return to step code
FF62: 68      PTCH93 PLA    ;not at phase A, restore
FF63: A8      TAY      ;Y
FF64: 68      PLA      ;and
FF65: AA      TAX      ;X
FF66: E64A    INC STEPS   ;count steps to do
FF68: AE001C  LDX DSKCNT  ;get motor phase
FF6B: CA      DEX      ;update it
FF6C: 4C38FA  JMP PPPPPP  ;and return
```

This concludes the differences between the 1541 325302-01 + 901229-06 and 251968-01 ROMs.

The second ROM revision for the 1541B introduced quite a lot of changes and can be regarded as a major effort to settle all known issues. The second patch area in the range \$C002-\$C0FF was used for the first time with this ROM revision.

The changes in ROM code were present in the entire ROM area covering \$C000-\$FFFF (ROM 251968-02). The changes that occurred between ROMs 901229-06 and 251968-01 were retained unchanged, apart from the 8 ms periodic interrupt that reverted back to 15 ms.

ROMs involved: 251968-01 and 251968-02

C057: AA	TAX	C057: 434F50+	***** ;* Add copyright for legal types and * ;* thieves * ***** CRIGHT .BY 'COPYRIGHT (C)1985 COMMODORE' .BY ' ELECTRONICS, LTD.', \$0D .BY 'ALL RIGHTS RESERVED ", \$0D
up			
to			
C098: AA	TAX		

Difference #2: Fix NODRV crossing a page.

The variable NODRV is used five times in the entire code, this change therefore yields five patches in total.

```
C1AD: A57F      SCREN1 LDA DRVNUM ;get current drive#
C1AF: 8D8E02      STA LSTDRV ;as list drive
C1B2: AA          TAX          ;and as index
C1B3: A900      LDA #0          ;set current drive status
C1B5: 95FF      STA NODRV,X ;!BUG: fails with drive 1
C1B7: 20BDC1     JSR CLRCB
```

```
FFB8: AA          TAX
FFB9: AA          TAX
FFBA: AA          TAX
FFBB: AA          TAX
FFBC: AA          TAX
FFBD: AA          TAX
FFBE: AA          TAX
FFBF: AA          TAX
```

```
C65D: A000      LDY #0
C65F: A67F      AUT01 LDX DRVNUM ;get drive number as index
C661: 98          TYA          ;get status
C662: 95FF      STA NODRV,X ;set status
C664: D003      RTCH20 BNE AUTO2
C666: 2042D0     JSR INITDR
C669: A67F      AUT02 LDX DRVNUM ;get drive number as index
C66B: B5FF      LDA NODRV,X ;!BUG drive1: get status
C66D: 60          RTS          ;and exit
```

```
FFAC: AA          TAX
FFAD: AA          TAX
FFAE: AA          TAX
FFAF: AA          TAX
```

```
FFC0: AA          TAX
FFC1: AA          TAX
FFC2: AA          TAX
FFC3: AA          TAX
FFC4: AA          TAX
FFC5: AA          TAX
FFC6: AA          TAX
```

```
C1AD: A57F      SCREN1 LDA DRVNUM ;get current drive#
C1AF: 8D8E02      STA LSTDRV ;as list drive#
C1B2: AA          TAX          ;and as index
C1B3: 4CB8FF     JMP PTCH43 ;! changed: NODRV is abs.
C1B6: EA          NOP          ;needed because of patch
C1B7: 20BDC1     RTCH43 JSR CLRCB
```

```
;*****
;* PTCH43:                                     *
;*                                             *
;* Change NODRV to absolute address           *
;*                                             *
;* (because of X indexing)                     *
;*                                             *
;*****
```

```
FFB8: A900      PTCH43 LDA #0          ;clear

FFBA: 9DFF00      .BY $9D,$FF,$00      ;write NODRV,X
                                           ;ABSOLUTE

FFBD: 4CB7C1      JMP RTCH43 ;and return
```

```
C65D: A000      LDY #0
C65F: A67F      AUT01 LDX DRVNUM ;get drive number
C661: 4CC0FF     JMP PTCH44 ;! changed

C664: D003      RTCH44 BNE AUTO2
C666: 2042D0     JSR INITDR
C669: A67F      AUT02 LDX DRVNUM ;get drive number as index
C66B: 4CACFF     JMP PTCH50 ;! changed:
C66D: 60          RTS          ;and exit (never executed)
```

```
;*****
;* PTCH50:                                     *
;*                                             *
;* Change NODRV to absolute address           *
;*                                             *
;* (because of X indexing)                     *
;*                                             *
;*****
```

```
FFAC: BDFF00     PTCH50 .BY $BD,$FF,$00 ;read NODRV,X
                                           ;ABSOLUTE
```

```
FFAF: 60          RTS
;*****
;* PTCH44:                                     *
;*                                             *
;* Change NODRV to absolute address           *
;*                                             *
;* (because of X indexing)                     *
;*                                             *
;*****
```

```
FFC0: 98          PTCH44 TYA
FFC1: 9DFF00      .BY $9D,$FF,$00      ;write NODRV,X
                                           ;ABSOLUTE

FFC4: 4C64C6      JMP RTCH44
```

```

D06A: A67F      LDX DRVNUM ;get drive# as index
D06C: 9D0101    STA DSKVER,X
D06F: A900      LDA #0
D071: 951C      STA WPSW,X ;clear WP switch
D073: 95FF      STA NODRV,X ;!BUG: fails with drive 1
D075: 203AEF    JSR SETBPT

```

```

FFC7: AA      TAX
FFC8: AA      TAX
FFC9: AA      TAX
FFCA: AA      TAX
FFCB: AA      TAX
FFCC: AA      TAX
FFCD: AA      TAX
FFCE: AA      TAX

```

```

F011: A56F    LF011 LDA TEMP
F013: 48      PHA
F014: A570    LDA T0+1
F016: 48      PHA
F017: A67F    PTCH52 LDX DRVNUM ;get drive# as index
F019: B5FF    LDA NODRV,X ;!BUG drive1: get status
F01B: F005    RTCH52 BEQ LF022

```

```

FFB0: AA      TAX
FFB1: AA      TAX
FFB2: AA      TAX
FFB3: AA      TAX
FFB4: AA      TAX
FFB5: AA      TAX
FFB6: AA      TAX
FFB7: AA      TAX

```

```

D06A: A67F      LDX DRVNUM ;get drive# as index
D06C: 9D0101    STA DSKVER,X
D06F: A900      LDA #0
D071: 4CC7FF    JMP PTCH51 ;! changed: NODRV is abs.
D074: EA        NOP ;needed because of patch
D075: 203AEF    RTCH51 JSR SETBPT

```

```

;*****
;* PTCH51:
;*
;* Change NODRV to absolute address
;*
;* (because of X indexing)
;*
;*****

```

```

FFC7: 951C    PTCH51 STA WPSW,X ;clear WP switch

FFC9: 9DFF00    .BY $9D,$FF,$00 ;write NODRV,X
;ABSOLUTE

FFCC: 4C75D0    JMP RTCH51 ;and return

```

```

F011: A56F    LF011 LDA TEMP
F013: 48      PHA
F014: A570    LDA T0+1
F016: 48      PHA
F017: 4CB0FF    JMP PTCH52 ;! changed: NODRV is abs.
F01A: EA        NOP ;needed because of patch
F01B: F005    RTCH52 BEQ LF022

```

```

;*****
;* PTCH52:
;*
;* Change NODRV to absolute address
;*
;* (because of X indexing)
;*
;*****

```

```

FFB0: A67F    PTCH52 LDX DRVNUM ;get drive# as index

FFB2: BDFF00    .BY $BD,$FF,$00 ;read NODRV,X
;ABSOLUTE

FFB5: 4C1BF0    JMP RTCH52 ;and return

```

Note that four of the five changes occur in the first half of the ROM, but the patches for these instances are all located in the second patch area. This is consistent with the fact that there is only one ROM device. In the two 8 kbyte ROM era, the four patches would have resided in the ROM for \$C000-\$DFFF. This would result in the possibility of only changing one ROM would still yield a working system; however this practice would not fix the problem being solved here. The approach used here is therefore technically correct.

Clarification: The IEEE-488 dual floppy drives of the past used two consecutive zero page RAM locations to flag whether or not a drive was in use. The two RAM locations were defined as variable NODRV with a length of two bytes: one for each drive. Getting the active flag involved loading register X with the drive number and getting the status with an LDA NODRV,X instruction. Over time some variables were moved around and some were added, with the result that in the V(I)C-1540 and 1541 NODRV ended up at locations \$FF and \$100, apparently without anybody noticing. When the aforementioned instruction LDA NODRV,X is assembled with NODRV defined at \$FF, the assembler will use zero page addressing.

In this particular instance this introduces a bug that is partly responsible for the infamous save-with-replace bug. When using zero page addressing the 6502 CPU will not cross a page when indexing (whether this is a bug or a feature is debatable; in later (CMOS) processors this behaviour was retained). In this case, when X=1 the processor would load the contents of location \$00 and not that of location \$100 which would be correct. The bug was fixed by changing the LDA NODRV,X instruction to use absolute indexed addressing. In practice however X=1 would never occur, as the 1541 only has one drive mechanism. This leads to the conclusion that the problem was probably discovered during the development of the dual drive 1541D which was never released. Anyway, the code is now bomb proof regarding this aspect...

The reason this fix is part of the repairing of the infamous save-with-replace bug stems from the fact that the DOS in the 1541 uses 5 sector buffers and upon power up, still reserves a buffer for each of the originally two drives of the 4040. Of the 5 buffers present, initially one is reserved for and assigned to the non-existent drive 1. In practice more than one file may be open on a drive, each file requiring one buffer. In situations where 4 buffers are in use opening another file or requesting a save-with replace on the same drive will cause the drive to run out of available buffers. Except there is still a buffer (the one reserved for the non-existent drive one) available. The DOS has code to check if there are buffers available because they are assigned to the other drive, which can be not in use. If the latter is the case, the available buffer assigned to the other drive will be used; this is called stealing a buffer. During this process the in-use status of drive one needs to be determined. As explained above, this goes wrong in previous ROM revisions, causing the save-with-replace bug, but only in very particular circumstances.

Difference #3: Fix for block read (B-R).

```
CD8C: 2064D4 BW20 JSR DRTWRT ;write block
CD8F: 68 PLA
CD90: AA TAX
CD91: 20CDED3 JSR RNDGET
CD94: 4C94C1 JMP ENDCMD
```

```
C024: AA TAX
C025: AA TAX
C026: AA TAX
C027: AA TAX
C028: AA TAX
```

```
CD8C: 2064D4 BW20 JSR DRTWRT ;write block
CD8F: 68 PLA
CD90: AA TAX
CD91: 2024C0 JSR PTCH15 ;!: load Y with LINDX
; (fix for block read)
CD94: 4C94C1 JMP ENDCMD
```

```
;*****
;* PTCH15: *
;* *
;* Load Y with LINDX *
;* *
;*****
C024: A482 PTCH15 LDY LINDX ;! changed: get LINDX
C026: 4CDED3 JMP RNDGET
```

Difference #4: Check for reportable errors after stealing a buffer.

```
D363: B500 STL40 LDA JOBS,X ;get job status
D365: 30FC BMI STL40 ;if not finished, wait
D367: C902 CMP #2 ;error# >2?
D369: 9007 BCC STL50 ;if not, proceed
D36B: A66F RTCH54 LDX T0
```

```
FF91: AA TAX
```

up
to

```
FF9C: AA TAX
FF9D: AA TAX
FF9E: AA TAX
```

```
D363: B500 STL40 LDA JOBS,X ;get job status
D365: 30FC BMI STL40 ;if not finished, wait
D367: 4C91FF JMP PTCH54 ;!: check for errors
D36A: EA NOP ;needed because of patch
D36B: A66F RTCH54 LDX T0
```

```
;*****
;* PTCH54: *
;* *
;* Check if reportable errors present *
;* *
;*****
FF91: C902 PTCH54 CMP #2 ;error# >2?
FF93: 9007 BCC PTH541 ;if not, proceed
FF95: C90F CMP #$0F ;! added: else if error 15
FF97: F003 BEQ PTH541 ;proceed
FF99: 4C6BD3 JMP RTCH54 ;else return reporting
;error
FF9C: 4C73D3 PTH541 JMP STL50
```

Difference #5: Set and clear micro stepping flag when recovering from read problems.

```
D60D B9DBFE LDA OFFSET,Y
D610: 2076D6 JSR HEDOFF ;move head
D613: EE9902 INC EPTR
D616: 20A6D6 JSR DOREC
D619: C902 CMP #2 ;error code < 2?
D61B: 9008 BCC LD625
D61D: AC9902 LDY EPTR
D620: B9DBFE LDA OFFSET,Y
D623: D0DB BNE LD600
D625: AD9A02 LD625 LDA TOFF
D628: 2076D6 JSR HEDOFF ;move head
```

```
FF84: AA TAX
```

up
to

```
FF90: AA TAX
```

```
D60D B9DBFE LDA OFFSET,Y
D610: 2084FF JSR PTCH12 ;!: set micro stepping flg
D613: EE9902 INC EPTR
D616: 20A6D6 JSR DOREC
D619: C902 CMP #2 ;error code < 2?
D61B: 9008 BCC LD625
D61D: AC9902 LDY EPTR
D620: B9DBFE LDA OFFSET,Y
D623: D0DB BNE LD600
D625: AD9A02 LD625 LDA TOFF
D628: 2089 FF JSR PTCH13 ;!: clr micro stepping flg
```

```
;*****
```

```
;* PTCH12: *
```

```
;* *
```

```
;* Set micro stepping flag *
```

```
;* *
```

```
;*****
```

```
FF84: 857B PTCH12 STA ADRSED ;set micro stepping flag
```

```
FF86: 4C76D6 JMP HEDOFF ;and move head
```

```
;*****
```

```
;* PTCH13: *
```

```
;* *
```

```
;* Clear micro stepping flag *
```

```
;* *
```

```
;*****
```

```
FF89: 2076D6 PTCH13 JSR HEDOFF ;move head
```

```
FF8C: A900 LDA #0 ;and
```

```
FF8E: 857B STA ADRSED ;clear micro stepping flag
```

```
FF90: 60 RTS ;then exit
```

Difference #6: Improve initializing variables when opening a file.

DCB6: A682	INITP	LDX LINDX		DCB6: A682	INITP	LDX LINDX	
DCB8: B5A7		LDA BUF0,X		DCB8: B5A7		LDA BUF0,X	
DCBA: 0A		ASL A		DCBA: 0A		ASL A	;get next bit
DCBB: A8		TAY	;	DCBB: 3006		BMI INITP1	;! added: if clear
DCBC: A902		LDA #\$02	;	DCBD: A8		TAY	;save current byte in Y
DCBE: 999900		STA BUFTAB,Y	;	DCBE: A902		LDA #\$02	;
DCC1: B5AE		LDA BUF1,X	;	DCC0: 999900		STA BUFTAB,Y	;
DCC3: 0980		ORA #\$80	;	DCC3: B5AE	INITP1	LDA BUF1,X	;
DCC5: 95AE		STA BUF1,X	;	DCC5: 0980		ORA #\$80	;set MSbit
DCC7: 0A		ASL A	;	DCC7: 95AE		STA BUF1,X	;
DCC8: A8		TAY	;	DCC9: 0A		ASL A	;get next bit
				DCCA: 3006		BMI INITP2	;! added:
DCC9: A902		LDA #\$02	;	DCCC: A8		TAY	;save A in Y as index
DCCB: 999900		STA BUFTAB,Y	;	DCCD: A902		LDA #\$02	;set
DCCE: A900		LDA #0	;clear	DCCF: 999900		STA BUFTAB,Y	;
DCD0: 95B5		STA NBKL,X	;	DCD2: A900	INITP2	LDA #0	;clear
DCD2: 95BB		STA NBKH,X	;	DCD4: 95B5		STA NBKL,X	;
DCD4: A900		LDA #0	;	DCD6: 4C29C0		JMP PTCH41	;! changed:
DCD6: 9D4402		STA LSTCHR,X	;	DCD9: EA		NOP	;(never executed)
DCD9: 60		RTS	;and exit				

							;* PTCH41: *
							;* *
							;* Complete INITP routine *
							;* *

C029: AA		TAX		C029: 95B5	PTCH41	STA NBKL,X	;! done already
up				C02B: 95BB		STA NBKH,X	
to				C02D: A900		LDA #\$00	;clear
				C02F: 9D4402		STA LSTCHR,X	
C032: AA		TAX		C032: 60		RTS	;and exit

Note that the patch is in the first patch area, although it would fit in the second one.

Another notable fact is that the patched code is functionally exactly the same as the unpatched code but requires more clock cycles to achieve the same result; the patch is needed because of added code before it. PTCH41 starts with executing the already done STA NBKL,X a second time, followed by the instructions that were originally already there. The LDA #0 (originally at \$DCD4, patched at \$C02D) is superfluous; the accumulator already contains 0 at this point because of the same instruction executed two/three instructions previously. PTCH41 can therefore be shortened:

```
*****
;* PTCH41 (improved)      *
;*                        *
;* Complete INITP routine *
;*                        *
*****
C029: 95BB PTCH41 STA NBKL,X
C02B: 9D4402 STA LSTCHR,X ;clear LSTCHR
C02E: 60 RTS ;and exit
```

Because the patch is a subroutine, the required RTS and extra stack operations also require extra cycles.

Difference #7: Fix HEXDEC routine: no interrupts and restore state of D-flag.

```
E69B: AA      HEXDEC TAX      ;get number to convert
E69C: A900    LDA #$00      ;clear result register
E69E: F8      SED          ;set decimal mode
E69F: E000    HEX0    CPX #0  ;if result zero, now
E6A1: F007    BEQ HEX5      ;we're done
                        ;! BUG: decimal mode remains set
```

```
C033: AA      TAX
```

up
to

```
C044: AA      TAX
C045: AA      TAX
C046: AA      TAX
```

```
E69B: AA      HEXDEC TAX
E69C: 4C33C0  JMP PTCH67    ;! changed: retain D-flag

E69F: E000    HEX0    CPX #0  ;if done (never ex.)
E6A1: F007    BEQ HEX5      ;proceed (never executed)
```

```
*****
;* PTCH67:                                *
;*                                         *
;* Complete HEXDEC routine                *
;*                                         *
*****
```

```
C033: 08      PTCH67 PHP      ;save flags
C034: 78      SEI          ;no interrupts
C035: A900    LDA #$00      ;clear result register
C037: F8      SED          ;set decimal mode
C038: E000    PTH671 CPX #0  ;if input is 0 now,
C03A: F007    BEQ PTH672    ;we're done
C03C: 18      CLC          ;prepare for add
C03D: 6901    ADC #1        ;increment A
C03F: CA      DEX          ;count tens
C040: 4C38C0  JMP PTH671    ;and loop
C043: 28      PTH672 PLP      ;restore flags (incl. D,I)
C044: 4CAAEE  JMP HEX5      ;and continue
```

Note that the patch is in the first patch area and does not fit in the second one, which is almost full. Clearly the new approach in the use of the two patch areas is to use the correct one until it is full, then use the other (first) one.

Difference #8: Test for ATN at end of reset code.

```
E780: 60      BOOT  RTS      ;return immediately
E781: EA      NOP
E781: EA      NOP
```

```
E780: 4C59EA  BOOT  JMP TSTATN ;! changed: test for ATN
```

Difference #9: Clear IRQ flag in VIA when servicing a new ATN while in ATN mode.

```
EA59: A57D      TSTATN LDA ATNMOD ;test if in ATN mode
EA5B: F006      BEQ TSTR50 ;if not, check if new ATN
EA5D: AD0018    LDA PB ;else
EA60: 1009      BPL TATN20 ;if ATN gone, do what we
                        ;have to do
EA62: 60        TSTRTN RTS ;else exit
EA63: AD0018    TSTR50 LDA PB ;still in ATN mode
EA66: 10FA      BPL TSTRTN ;if no ATN, exit
EA68: 4C5BE8    JMP ATNSRV ;else service ATN
                        ;!BUG: IRQ flag in 6522
                        ;is not cleared

EA6B: 4C D7 E8 TATN20 JMP ATNS20
```

```
FFA6: AA        TAX
FFA7: AA        TAX
FFA8: AA        TAX
FFA9: AA        TAX
FFAA: AA        TAX
FFAB: AA        TAX
```

```
EA59: A57D      TSTATN LDA ATNMOD ;test if in ATN mode
EA5B: F006      BEQ TSTR50 ;if not, check if new ATN
EA5D: AD0018    LDA PB ;else
EA60: 1009      BPL TATN20 ;if ATN gone, do what we
                        ;have to do
EA62: 60        TSTRTN RTS ;else exit
EA63: AD0018    TSTR50 LDA PB ;still in ATN mode
EA66: 10FA      BPL TSTRTN ;if no ATN, exit
EA68: 4CA6FF    JMP PTCH30 ;! changed: else clear
                        ;IRQ flag and service ATN

EA6B: 4C D7 E8 TATN20 JMP ATNS20
```

```
                        ;*****
                        ;* PTCH30: *
                        ;* *
                        ;* Clear IRQ flags in VIA before *
                        ;* servicing new ATN *
                        ;*****
FFA6: 2C0118    PTCH30 BIT PA1 ;clear IRQ flag

FFA9: 4C5BE8    JMP ATNSRV ;do ATN command
```

Difference #10: Execute reset routine with interrupts blocked.

```
EB22: A245      DIAGOK LDX #$45 ;get top of stack
EB24: 9A        TXS ;set stack pointer
EB25: AD001C    LDA LEDPRT ;get LED port
```

```
FF9F: AA        TAX
FFA0: AA        TAX
FFA1: AA        TAX
FFA2: AA        TAX
FFA3: AA        TAX
FFA4: AA        TAX
FFA5: AA        TAX
```

```
EB22: 4C9FFF    DIAGOK JMP PTCH31 ;! changed: add SEI,
                        ;init SP
EB25: AD001C    RTCH31 LDA LEDPRT ;get LED port
```

```
                        ;*****
                        ;* PTCH31: *
                        ;* *
                        ;* block interrupts before initializing *
                        ;* stack pointer *
                        ;* *
                        ;*****
FF9F: 78        PTCH31 SEI ;added: no interrupts
FFA0: A245      LDX #$45 ;initialize

FFA2: 9A        TXS ;stack pointer
FFA3: 4C25EB    JMP RTCH31 ;and proceed
```

Difference #11: At the end of the self test, recalibrate head to outermost track.

EBBF: 20FACE	JSR LRUINT	;initialize the LRU table	EBBF: 20FACE	JSR LRUINT	;initialize the LRU table
EBC2: 206FFF	JSR PTCH10	;perform a bump	EBC2: 2073FF	JSR PTCH10	;PATCH: perform a bump

			* PTCH10:		
			*		
			* Move head to track zero at end of		
			* device initialization		
			*		

FF73: AA	TAX		FF73: 2059F2	PTCH10 JSR CNTINT	;controller init
FF74: AA	TAX				
FF75: AA	TAX				
FF76: AA	TAX		FF76: A901	LDA #\$01	;command is 1 byte long
FF77: AA	TAX				
FF78: AA	TAX		FF78: 8506	STA HDRS	
FF79: AA	TAX				
FF7A: AA	TAX		FF7A: A9C0	LDA #\$C0	;get bump command
FF7B: AA	TAX				
FF7C: AA	TAX		FF7C: 8500	STA JOBS	;set it
FF7D: AA	TAX				
FF7E: AA	TAX		FF7E: 60	RTS	;and exit

This difference is listed because PTCH10 was moved. The code itself is unchanged.

Difference #12: Set drive active before starting format.

EE19: 2901	N101	AND #\$01	;get drive#	EE19: 2901	N101	AND #\$01
EE1B: 857F	STA DRVNUM	;set it		EE1B: 857F	STA DRVNUM	
EE1D: 2000C1	JSR SETLDS	;update drive LEDs		EE1D: 207FFF	JSR PTCH11	;! changed: add clear
						;NODRV (with bug)
EE20: A57F	LDA DRVNUM			EE20: A57F	LDA DRVNUM	

				;* PTCH11:		
				*		
				;* clear NODRV		

FF7F: AA	TAX			FF7F: 85FF	PTCH11 STA NODRV	;clear NODRV
FF80: AA	TAX					;!BUG: drive#
FF81: AA	TAX			FF81: 4C00C1	JMP SETLDS	;and set LEDs
FF82: AA	TAX					
FF83: AA	TAX					

This patch has serious problems in the context of this ROM revision. Difference #2 is a major effort to fix the potential page cross problem with NODRV. Yet the patch:

1. repeats the same error by using zero page addressing;
2. forgets to index for drive 1 support;
3. fails when drive 1 would be actually used, because NODRV for drive 0 (!) would be set to the wrong value of 1.

Amazingly, the rest of the NEW routine handles drive 1 correctly. When will we ever learn...

A correct patch would be:

```
FF7F: AA      PTCH11 TAX      ;get drive# as index
FF80: A900      LDA #0      ;set current drive inactive
FF82: 9DFF00     .BY $9D,$FF,$00 ;write NODRV,X
                                ;ABSOLUTE
FF85: 4C00C1     JMP SETLDS  ;and set LEDs
```

Difference #13: If less than 3 blocks free, report 1 block free.

The purpose of this change is to fix the disc full bug. When three or less free blocks remain, the DOS starts reporting one block free. The idea behind this is to have sufficient blocks free (if possible) in order to be able to close a potentially open write file successfully. A side effect is that under these circumstances, a directory listing will show an incorrect number of free blocks, a condition that eventually corrects itself or can be rectified by validating the disc.

EFBD:	BDFC02	USE20	LDA NDBH,X	; get blocks free hi	EFBD:	BDFC02	USE20	LDA NDBH,X	;get blocks free hi
EFC0:	D00C		BNE USERTS	;if not zero, exit	EFC0:	D00C		BNE USERTS	;if not zero, exit
EFC2:	BDFC02		LDA NDBL,X	;get blocks free lo	EFC2:	BDFC02		LDA NDBL,X	;get blocks free lo
EFC5:	C903		CMP #\$03	;if less than 3	EFC5:	4C47C0		JMP PCTH66	;! changed: keep at least
EFC7:	B005		BCS USERTS						;2 blocks available
EFC9:	A972		LDA #\$72	;get error number (Disc	EFC8:	EA		NOP	;needed because of patch
				;full)	EFC9:	A972		LDA #\$72	;get error number (Disc
EFCB:	20C7E6		JSR ERRMSG	;and report it					;full) (never executed)
EFCE:	60	USERTS	RTS	;then exit	EFCB:	20C7E6		JSR ERRMSG	;and report it(never ex.)
					EFCE:	60	USERTS	RTS	;then exit
									;*****
									;* PCTH66: *
									;*
									;* If less than 3 blocks free, report *
									;* only one block free to keep 2 *
									;* sectors available for a successful *
									;* file close. *
									;*
									;*****
C047:	AA		TAX		C047:	C903	PCTH66	CMP #3	;if 3 or less blocks free
up					C049:	B005		BCS PTH661	;report 1 block free
to					C04B:	A972		LDA #\$72	;else
					C04D:	20C7E6		JSR ERRMSG	;report disc full
C052:	AA		TAX		C050:	A901	PTH661	LDA #1	;get remaining blocks
					C052:	60		RTS	;and exit

Difference #14: Reverse change of periodic interrupt every 8 ms.

The reason given in the release documentation for this reversal is compatibility; some software in the field apparently uses the 15 ms period between IRQs for timing purposes. (This is bad programming practice: one should not rely on internals of an operating environment).

F26C:	8D0C1C		STA PCR2		F26C:	8D0C1C		STA PCR2	
F26F:	A941		LDA #\$41	;cont IRQ, latch mode	F26F:	A941		LDA #\$41	;cont IRQ, latch mode
F271:	8D0B1C		STA PCR2		F271:	8D0B1C		STA PCR2	
F274:	A900		LDA #\$00	;get timer1 lo byte	F274:	A900		LDA #\$00	;get timer1 lo byte
F276:	8D061C		STA T1LL2	;set it	F276:	8D061C		STA T1LL2	;set it
F279:	A920		LDA #\$20	;8ms IRQ	F279:	A93A		LDA #\$3A	;! changed: 15ms IRQ
F27B:	8D071C		STA T1HL2	;set timer 2 hi	F27B:	8D071C		STA T1HL2	;set timer 2 hi
F27E:	8D051C		STA T1HC2	;get 6522's attention	F27E:	8D051C		STA T1HC2	;get 6522's attention
F281:	A97F		LDA #\$7F	;clear all IRQ sources	F281:	A97F		LDA #\$7F	;clear all IRQ sources

This change reverses Difference #3 in ROM 251968-01.

Difference #15: Expand track zone table to 40 tracks.

The drive mechanism in the 1541 actually has 40 physical and usable tracks, not 35 as in the 4040. Some software accessed tracks 36-40 (or even up to 42). The problem is that the zone table stops at track 35, so the bit density would not be set correctly for tracks over 35.

F33C:	A204	G0TU	LDX #4	;set track, sector	F33C:	A204	G0TU	LDX #4	;set track, sector
F33E:	B132		LDA (HDPNT),Y	;get track#	F33E:	B132		LDA (HDPNT),Y	
F340:	8540		STA TRACC	;save it	F340:	8540		STA TRACC	
F342:	DDD6FE	LF342	CMP TRKNUM-1,X	;compare	F342:	DD52C0	LF342	CMP TRACKN-1,X	;! change: new table
F345:	CA		DEX	;get zone number	F345:	CA		DEX	
					;*****				
					;* New track zone table *				
					;*****				
C053:	AA		TAX		C053:	29	TRACKN	.BY 41	;! changed: track 36->41
C054:	AA		TAX		C054:	1F		.BY 31	
C055:	AA		TAX		C055:	19		.BY 25	
C056:	AA		TAX		C056:	12		.BY 18	

Note that the old unmodified table is left in place rather than changing one byte in it. This approach ensures greater compatibility in cases the original table would be used in a user program, but also renders the new table useless. This fact stems from the fact that the LCC uses the first byte of the table to check if a track number is within range. For this reason, the first byte of the original table has two labels; the first representing the maximum track number, the second the table itself. The table was changed but the address for the maximum track number in the LCC was not, still using the old value of 36. Despite the new table, the LCC will still report an illegal track number for track numbers 36 and higher, rendering the new table useless.

Commodore did not use all 40 physical tracks in the 1541 giving it more storage space while at the same time allowing 35 track formatted discs to be used for compatibility with older drives; a gap that was successfully exploited by other software.

Difference #16: Change test for minimum gap2 from 4 to 2.

FC17:	A200		LDX #\$00		FC17:	A200		LDX #\$00	
FC19:	38	DS20	SEC		FC19:	38	DS20	SEC	
FC1A:	E543		SBC SECTR		FC1A:	E543		SBC SECTR	
FC1C:	B003		BCS DS22		FC1C:	B003		BCS DS22	
FC1E:	88		DEY		FC1E:	88		DEY	
FC1F:	3003		BMI DS30		FC1F:	3003		BMI DS30	
FC21:	E8	DS22	INX		FC21:	E8	DS22	INX	
FC22:	D0F5		BNE DS20		FC22:	D0F5		BNE DS20	
FC24:	8E2606	DS30	STX DTRCK		FC24:	8E2606	DS30	STX DTRCK	
FC27:	E002		CPX #4	;gap2	FC27:	E002		CPX #2	;! changed: gap2 was 4
FC29:	B005		BCS DS32		FC29:	B005		BCS DS32	

Note: the value 2 for gap2 is the value originally used in the 1540/2031/4040.

Difference #17: Clear disc before formatting using less bytes.

FCAA: A900	LDA #\$00			FCAA: A900	LDA #\$00	;init counter
FCAC: 8532	STA HDRPNT			FCAC: 8532	STA HDRPNT	
FCAE: 200EFE	JSR CLEAR			FCAE: 2002C0	JSR CLEAR	;! new CLEAR routine
FCB1: A9FF	LFCB1 LDA #\$FF			FCB1: A9FF	WRTSYN LDA #\$FF	
FE0E: AD0C1C	CLEAR LDA PCR2		;enable write	C002: AD0C1C	CLEAR LDA PCR2	;enable write
FE11: 291F	AND #\$1F			C005: 291F	AND #\$1F	;clear bit 5-7
FE13: 09C0	ORA #\$C0			C007: 09C0	ORA #\$C0	;set bit 6 & 7
FE15: 8D0C1C	STA PRC2			C009: 8D0C1C	STA PCR2	;
FE18: A9FF	LDA #\$FF			C00C: A9FF	LDA #\$FF	;make port an output
FE1A: 8D031C	STA DDR2			C00E: 8D031C	STA DDRA2	
FE1D: A955	LDA #\$55			C011: A955	LDA #\$55	;write a 1F pattern
FE1F: 8D011C	STA DATA2			C013: 8D011C	STA DATA2	
FE22: A228	LDX 40		;get outer loop count	C016: A203	LDX #3	!; get outer loop count
FE24: A000	LDY #0		get inner loop cnt (256)	C018: A000	LDY #0	;get inner loop cnt (256)
FE26: 50FE	CLER10 BVC CLER10		;wait for controller	C01A: 50FE	CLER11 BVC CLER11	;wait for controller
FE28: B8	CLV		;clear controller ready	C01C: B8	CLV	;clear controller ready
FE29: 88	DEY		;count inner loop	C01D: 88	DEY	;count inner loop
FE2A: D0FA	BNE CLER10		;if not done, loop	C01E: D0FA	BNE CLER11	;if not done, loop
FE2C: CA	DEX		count outer loop	C020: CA	DEX	;count outer loop
FE2D: D0F7	BNE CLER10		;if not done, loop	C021: D0F7	BNE CLER11	;if not done, loop
FE2F: 60	RTS		;else exit	C023: 60	RTS	;else exit

Note: the routine CLEAR on the left is still present in the ROM unchanged, and has been listed for comparison reasons. The only difference is the number of bytes the disc is cleared with: it used to be $256 \times 40 = 10240$ bytes (more than two tracks), the new value is $256 \times 3 = 768$ bytes (about two and a half disc sectors).

This concludes the differences between the 1541 251968-01 and 251968-02 ROMs.

1541-II, 251968-03 ROM

This is the first and effectively only ROM version for the 1541-II. It is generally speaking the last ROM revision for the 1541B (ROM 251968-02) with the support for the track 0 sensor removed. The changed copyright statement and some patches have been moved to different locations, but their code has not changed; only changes in code have been listed below.

The changes in ROM code were present in the entire ROM area covering \$C000-\$FFFF (251968-03). The changes that occurred between ROMs 251958-01 and 251968-02 were retained largely unchanged, apart from the removal of the support for the track 0 sensor (including control of the micro stepping flag) and return to the old zone table. The test using gap2 now uses the old value for gap2 (4) again. The net result is no bug fixes and return to values in use with the majority of ROMs for the 1541 in the world at this point in time.

The ROM 251968-02 will also work in the 1541-II, because its main board has the VIA input PA0 used for the track 0 sensor on the 1541B connected to ground, defeating the sensor. Commodore probably grounded the input to be able to use the 251968-02 ROM for the 1541B in the 1541-II. The most probable cause for this is that by the time the production of the 1541-II started, the 251968-03 ROM code was not ready yet. A 1541-II equipped with a 251968-02 ROM will bump on power up.

Code differences compared to the ROM 251968-02

ROMs involved: 251968-02 and 251968-03

Difference #1: Copyright statement changed and moved.

Space previously occupied by now unused or moved patches.

C002:	434F+	.BY 'COPYRIGHT (C)1982,1985,1987'
		.BY ' COMMODORE ELECTRONICS, LTD.'
C039:	0D	.BY \$0D
C03A:	414C+	.BY 'ALL RIGHTS RESERVED'
C04D:	0D	.BY \$0D

Difference #2: Set/clear micro stepping flag removed.

```
D60D B9DBFE LDA OFFSET,Y
D610: 2084FF JSR PTCH12 ;!: set micro stepping flg
D613: EE9902 INC EPTR
D616: 20A6D6 JSR DOREC
D619: C902 CMP #2 ;error code < 2?
D61B: 9008 BCC LD625
D61D: AC9902 LDY EPTR
D620: B9DBFE LDA OFFSET,Y
D623: D0DB BNE LD600
D625: AD9A02 LD625 LDA TOFF
D628: 2089 FF JSR PTCH13 ;!: clr micro stepping flg
```

```
*****
;* PTCH12: *
;* *
;* Set micro stepping flag *
;* *
*****
FF84: 857B PTCH12 STA ADRSED ;set micro stepping flag
FF86: 4C76D6 JMP HEDOFF ;and move head
*****
;* PTCH13: *
;* *
;* Clear micro stepping flag *
;* *
*****
FF89: 2076D6 PTCH13 JSR HEDOFF ;move head
FF8C: A900 LDA #0 ;and
FF8E: 857B STA ADRSED ;clear micro stepping flag
FF90: 60 RTS ;then exit
```

```
D60D B9DBFE LDA OFFSET,Y
D610: 2076D6 JSR HEDOFF ;move head
D613: EE9902 INC EPTR
D616: 20A6D6 JSR DOREC
D619: C902 CMP #2 ;error code < 2?
D61B: 9008 BCC LD625
D61D: AC9902 LDY EPTR
D620: B9DBFE LDA OFFSET,Y
D623: D0DB BNE LD600
D625: AD9A02 LD625 LDA TOFF
D628: 2076D6 JSR HEDOFF ;move head
```

```
FF84: AA TAX
up
to
FF90: AA TAX
```

This change reverses Difference #5 introduced with the previous revision 251968-02.

Difference #3: Remove test for ATN at end of self test.

```
E780: 4C59EA BOOT JMP TSTATN ;test for ATN
```

```
E780: EA BOOT NOP ;! changed
E781: EA NOP ;! changed
E781: EA NOP ;! changed
```

This change reverses Difference #8 introduced with the previous ROM revision 251968-02. Note that the instruction at location \$E780 is now a NOP (it used to be an RTS before the test for ATN was added). The RTS at \$E7A2 now has this function after needlessly executing a number of NOPs.

Difference #4: VIA pin PA0 is an output again.

<pre>***** ;* RESET entry point * ***** EAA0: 78 DSKINT SEI ;no interrupts EAA1: D8 CLD ;clear decimal mode EAA2: A2FE LDX #\$FE ;PA0 is an input, ;rest output</pre>	<pre>***** ;* RESET entry point * ***** EAA0: 78 DSKINT SEI ;no interrupts EAA1: D8 CLD ;clear decimal mode EAA2: A2FF LDX #\$FF ;! changed: port A is ;all outputs</pre>
---	---

Note: this simply reverses the setting of PA0 to an input, but does not take into account that the PA0 pin is connected to ground on the 1541-II main board. As port A is not initialized properly (only the direction is set) the PA0 pin could be driven high, constituting in effect a short circuit and potentially destroying the internal VIA circuitry for PA0. This change should not have been made.

This change reverses Difference #1 introduced with ROM revision 251968-01.

Difference #5: Remove head recalibrate at the end of the self test.

<pre>EBBA: 851D STA WPSW+1 ;clear WP status drive 1 EBBC: 2063CB JSR USRINT ;initialize USER JMP EBBF: 20FACE JSR LRUINT ;initialize LRU EBC2: 206FFF JSR PTCH10 ;perform a bump EBC5: A922 LDA #<DIAGOK EBC7: 8565 STA VNMI ***** ;* PTCH10: * ;* * ;* Move head to track zero at end of * ;* device initialization * ;* * ***** FF6F: 2059F2 PTCH10 JSR CNTINT ;controller initialization FF72: A901 LDA #\$01 ;command length FF74: 8506 STA HDRS ;is one byte FF76: A9C0 LDA #\$C0 ;get bump command FF78: 8500 STA JOBS ;set it FF7A: 60 RTS ;and exit</pre>	<pre>EBBA: 851D STA WPSW+1 ;clear WP status drive 1 EBBC: 2063CB JSR USRINT ;initialize USER JMP EBBF: 20FACE JSR LRUINT ;initialize LRU EBC2: 2059F2 JSR CNTINT ;! changed: contrllr init EBC5: A922 LDA #<DIAGOK EBC7: 8565 STA VNMI FF6F: AA TAX up to FF7A: AA TAX</pre>
---	---

This change reverses Difference #2 introduced with the previous ROM revision 251968-01.

Difference #6: Revert to the previous track zone table.

```
F33C: A204    GOTU    LDX #4      ;set track, sector
F33E: B132          LDA (HDPNT),Y
F340: 8540          STA TRACC
F342: DD52C0  LF342  CMP TRACKN-1,X      ;new zone table
F345: CA          DEX

;*****
;* New track zone table      *
;*****

C053: 29      TRACKN .BY 41
C054: 1F          .BY 31
C055: 19          .BY 25
C056: 12          .BY 18
```

```
F33C: A204    GOTU    LDX #4      ;set track, sector
F33E: B132          LDA (HDPNT),Y
F340: 8540          STA TRACC
F342: DDD6FE  LF342  CMP TRKNUM-1,X      ;! changed
F345: CA          DEX

C053: AA          TAX
C054: AA          TAX
C055: AA          TAX
C056: AA          TAX
```

This changes reverses Difference #15 introduced with ROM revision 251968-02.

Difference #7: Remove check for track 0 when stepping out.

```
FA2E: A54A    DOSTEP LDA STEPS    ;get steps to do/direction
FA30: 1031    BPL STPIN    ;if stepping in, do that
FA32: 4C36FF STPOUT JMP PATCH9    ;check for track 0
FA35: EA      NOP          ;needed because
FA36: EA      NOP          ;of
FA37: EA      NOP          ;patch
FA38: 4C69FA PPPPPP JMP STP      ;
FA3B: A54A    SHORT LDA STEPS    ;get # of steps to do
```

```
*****
;* PATCH9:                                     *
;*                                           *
;* Check if at track 0                         *
;*                                           *
*****
```

```
FF36: 8A      PATCH9 TXA        ;save
FF37: 48      PHA              ;X
FF38: 98      TYA              ;and
FF39: 48      PHA              ;Y
FF3A: A201    LDX #$01         ;get outer loop count
FF3C: A064    PTCH91 LDY #100   ;get inner loop count
FF3E: AD0F18 PTCH92 LDA POTA1   ;get port A
FF41: CD0F18 CMP POTA1        ;if no change
FF44: D01C    BNE PTCH93
FF46: 88      DEY              ;count down (inner loop)
FF47: D0F5    BNE PTCH92       ;if inner not done, loop
FF49: CA      DEX              ;else adapt outer loop
FF4A: D0F0    BNE PTCH91       ;if not finished, loop
FF4C: 2901    AND #$01         ;else get port data
FF4E: F012    BEQ PTCH93       ;if not at track 0
FF50: AD001C LDA DSKCNT        ;get controller phase
FF53: 2903    AND #$03         ;bits
FF55: D00B    BNE PTCH93       ;if at phase A
FF57: 68      PLA              ;exit
FF58: A8      TAY              ;restoring
FF59: 68      PLA              ;Y
FF5A: AA      TAX              ;and X
FF5B: A900    LDA #0           ;clear
FF5D: 854A    STA STEPS        ;number of steps to do
FF5F: 4CBEFA JMP END33        ;and return to step code
FF62: 68      PTCH93 PLA        ;not at phase A, restore
FF63: A8      TAY              ;Y
FF64: 68      PLA              ;and
FF65: AA      TAX              ;X
FF66: E64A    INC STEPS        ;count steps to do
FF68: AE001C LDX DSKCNT        ;get motor phase
FF6B: CA      DEX              ;update it
FF6C: 4C38FA JMP PPPPPP        ;and return
```

```
FA2E: A54A    DOSTEP LDA STEPS    ;get steps to do/direction
FA30: 1031    BPL STPIN    ;if stepping in, do that
FA32: E64A    STPOUT INC STEPS    ;! changed: steps to do
FA34: AE001C LDX DSKCNT        ;get phase
FA37: CA      DEX              ;update it
```

```
FA38: 4C69FA JMP STP          ;
FA3B: A54A    SHORT LDA STEPS    ;get # of steps to do
```

```
FF36: AA      TAX
```

```
up
to
```

```
FF6C: AA      TAX
FF6D: AA      TAX
FF6E: AA      TAX
```

This change reverses Difference #4 introduced with ROM revision 251968-01.

Difference #8: Change test for minimum gap2 from 2 back to 4.

FC17:	A200		LDX #\$00		FC17:	A200		LDX #\$00	
FC19:	38	DS20	SEC		FC19:	38	DS20	SEC	
FC1A:	E543		SBC SECTR		FC1A:	E543		SBC SECTR	
FC1C:	B003		BCS DS22		FC1C:	B003		BCS DS22	
FC1E:	88		DEY		FC1E:	88		DEY	
FC1F:	3003		BMI DS30		FC1F:	3003		BMI DS30	
FC21:	E8	DS22	INX		FC21:	E8	DS22	INX	
FC22:	D0F5		BNE DS20		FC22:	D0F5		BNE DS20	
FC24:	8E2606	DS30	STX DTRCK		FC24:	8E2606	DS30	STX DTRCK	
FC27:	E002		CPX #2	;gap 2	FC27:	E002		CPX #4	;! changed: gap2 is 4
FC29:	B005		BCS DS32		FC29:	B005		BCS DS32	

This change reverses Difference #16 introduced with ROM revision 251968-02.

General remark: the location of a code patch (patch area one or two) is no longer consistent. Because of the removal of the support for the track 0 sensor, patch area two now has more space available. The patches still needed have remained in the patch area they were before although they all fit in patch area two.

This concludes the differences between the 1541B 251968-02 and 1541-II 251968-03 ROMs.

1541-II, 355649-01 ROM

This 1541-II ROM version is reported by some sources as being present in some 1541-II drives with Newtronics mechanisms. Analysis of the contents reveals the following:

The part \$C000-\$DFFF is equal to the ROM 325302-01.

The part \$E000-\$FFFF has the following features compared to ROM 251968-03:

1. No copyright statement;
2. NODRV problems are not fixed;
3. CLEAR routine is the first version;
4. Check for reportable errors after stealing a buffer is absent;
5. Fix to improve initializing variables when opening a file is absent;
6. HEXDEC routine has not been fixed;
7. TSTATN does not test for a 2nd ATN while processing current ATN;
8. Reset routine executed with interrupts enabled;
9. Format flag is not set before formatting a disc;
10. Disc full bug still present;
11. Gap1 has the new value of 9.

These features are a match with ROM 901229-05.

The ROM is therefore simply the contents of the older ROMs 325302-01 and 901229-05 combined in one 16 kbyte device, and brings nothing new (in fact it is a step back compared to ROM 251968-03).

Code differences compared to the ROMs 325302-01 & 901229-05

ROMs involved: 325302-01 + 901229-05 and 335640-01

There are no differences; all bytes are the same.

Remark: in all 1541 ROMs there is no code specific for a particular mechanism manufacturer; these differences were solved in the hardware. This ROM revision is therefore not specifically geared towards Newtronics drives.

Appendix A: ROM checksum of the original 1541 ROMs

The various ROMs have the following hex checksums:

Version	C1	C2	E1	E2	Model	ROM size
325302-01 + 901229-01	97	X	FD	X	1541	8 kbyte
325302-01 + 901229-02	97	X	10	X	1541	8 kbyte
325302-01 + 901229-03	97	X	0E	X	1541	8 kbyte
325302-01 + 901229-04	97	X	D0	X	1541	8 kbyte
325302-01 + 901229-05	97	X	3E	X	1541	8 kbyte
325302-01 + 901229-06	97	X	EC	X	1541	8 kbyte
251968-01	97	AA	79	C6	1541B	16 kbyte
251968-02	97	46	79	CA	1541B	16 kbyte
251968-03	97	E0	79	EB	1541-II	16 kbyte

X means in this case: not used as checksum correction byte. That byte usually has the value \$AA in the ROM.

ROM addresses are:

Code	Address	Remark
C1	\$C000	Value is always \$97
C2	\$C001	Unused in 1541's with 8k ROMs
E1	\$FEE6	Value is always \$79 with 16k ROMs
E2	\$FFE5	Unused in 1541's with 8k ROMs

1541's use two 8 kbyte ROMs; the 1541B and 1541-II use one 16 kbyte device.
